

# Documentação Técnica - RH Gestão de Funcionários

---

## Índice

1. [Visão Geral](#)
  2. [Arquitetura do Sistema](#)
  3. [Tecnologias Utilizadas](#)
  4. [Estrutura do Projeto](#)
  5. [Modelo de Dados](#)
  6. [API Backend](#)
  7. [Frontend](#)
  8. [Configuração e Instalação](#)
  9. [Detalhes Técnicos](#)
  10. [Fluxos de Trabalho](#)
- 

## Visão Geral

O **RH Gestão de Funcionários** é um sistema completo para gestão de funcionários e controle de custos de benefícios empresariais. O sistema permite:

- **Cadastro e gestão de funcionários:** CRUD completo com controle de admissão/demissão
- **Gestão de Vale Mercado:** Rateio de custos entre empresa e funcionários (95% empresa / 5% funcionários)
- **Gestão de Vale Refeição:** Controle de faturas e rateio (80% empresa / 20% funcionários) para duas filiais
- **Gestão de Plano de Saúde (Unimed):** Rateio de custos médicos por competência
- **Gestão de Convênios Médicos:** Controle de múltiplos prestadores (Laboratórios e Clínicas)

O sistema foi desenvolvido como um **MVP (Minimum Viable Product)** focado em funcionalidades essenciais de cadastro, edição, demissão e listagem com filtros avançados.

---

## Arquitetura do Sistema

O projeto segue uma arquitetura **monorepo** com separação clara entre backend e frontend:

```
rh-gestao-funcionarios/  
├── api/           # Backend (Node.js + Express + Prisma)  
├── web/           # Frontend (React + TypeScript + Vite)  
└── docker-compose.yml # Infraestrutura (MySQL + Adminer)
```

### Padrão Arquitetural

- **Backend:** API RESTful com Express.js
- **Frontend:** SPA (Single Page Application) com React

- **Banco de Dados:** MySQL 8.0 (via Docker)
- **ORM:** Prisma para modelagem e acesso aos dados
- **Validação:** Zod para validação de schemas

---

## Tecnologias Utilizadas

Backend ([api/](#))

| Tecnologia  | Versão  | Propósito                              |
|-------------|---------|----------------------------------------|
| Node.js     | 18+     | Runtime JavaScript                     |
| TypeScript  | ^5.5.4  | Linguagem tipada                       |
| Express     | ^4.19.2 | Framework web                          |
| Prisma      | ^5.22.0 | ORM e migrações                        |
| Zod         | ^3.23.8 | Validação de schemas                   |
| CORS        | ^2.8.5  | Controle de acesso CORS                |
| dotenv      | ^16.4.5 | Gerenciamento de variáveis de ambiente |
| ts-node-dev | ^2.0.0  | Hot reload em desenvolvimento          |

Frontend ([web/](#))

| Tecnologia | Versão  | Propósito                |
|------------|---------|--------------------------|
| React      | ^18.3.1 | Biblioteca UI            |
| TypeScript | ^5.5.4  | Linguagem tipada         |
| Vite       | ^5.4.6  | Build tool e dev server  |
| Axios      | ^1.7.4  | Cliente HTTP             |
| Recharts   | ^3.6.0  | Gráficos e visualizações |

Infraestrutura

| Tecnologia | Versão | Propósito                 |
|------------|--------|---------------------------|
| MySQL      | 8.0    | Banco de dados relacional |
| Docker     | -      | Containerização           |
| Adminer    | latest | Interface web para MySQL  |

---

## Estrutura do Projeto

Estrutura de Diretórios

```

rh-gestao-funcionarios/
├── api/                                     # Backend API
│   ├── src/
│   │   ├── lib/                           # Bibliotecas utilitárias
│   │   │   ├── format.ts                 # Funções de formatação
│   │   │   └── prisma.ts                 # Cliente Prisma singleton
│   │   ├── routes/                       # Rotas da API
│   │   │   ├── employees.ts              # CRUD de funcionários
│   │   │   ├── voucherMarket.ts         # Vale Mercado
│   │   │   ├── voucherMeal.ts           # Vale Refeição
│   │   │   ├── unimed.ts                 # Plano de Saúde Unimed
│   │   │   └── medicalConvenios.ts       # Convênios médicos
│   │   └── server.ts                     # Servidor Express
│   ├── prisma/
│   │   ├── schema.prisma                 # Schema do banco de dados
│   │   ├── migrations/                  # Migrações do Prisma
│   │   ├── seed.ts                       # Script de seed
│   │   └── seed.js                       # Script de seed (alternativo)
│   ├── data/
│   │   └── funcionarios.json             # Dados iniciais para seed
│   ├── .env.example                      # Exemplo de variáveis de ambiente
│   ├── package.json                      # Dependências do backend
│   └── tsconfig.json                     # Configuração TypeScript
├── web/                                   # Frontend React
│   ├── src/
│   │   ├── components/                  # Componentes React
│   │   │   └── Sidebar.jsx               # Barra lateral de navegação
│   │   ├── layout/                      # Componentes de layout
│   │   │   └── AppShell.jsx              # Shell da aplicação
│   │   ├── pages/                       # Páginas da aplicação
│   │   │   ├── EmployeesPage.tsx        # Página de funcionários
│   │   │   ├── ValeMercadoPage.tsx      # Página Vale Mercado
│   │   │   ├── ValeRefeicaoPage.tsx      # Vale Refeição Filial 01
│   │   │   ├── ValeRefeicaoFilial02Page.tsx # Vale Refeição Filial 02
│   │   │   ├── UnimedPage.tsx           # Página Unimed
│   │   │   ├── LaboratorioSantaCruzPage.tsx
│   │   │   ├── CentroDiagnosticoCapaoRasoPage.tsx
│   │   │   ├── PoliclinicaCapaoRasoPage.tsx
│   │   │   ├── PoliclinicaMansurPage.tsx
│   │   │   └── MedicalConvenioTemplate.tsx # Template para convênios
│   │   ├── services/                    # Serviços de API
│   │   │   ├── apiClient.js             # Cliente HTTP base
│   │   │   └── valeRefeicaoApi.js        # API específica Vale Refeição
│   │   ├── styles/                      # Estilos CSS
│   │   │   └── app.css                   # Estilos da aplicação
│   │   ├── api.ts                       # Tipos e cliente API
│   │   ├── App.tsx                      # Componente principal
│   │   ├── Shell.tsx                    # Shell wrapper
│   │   ├── main.tsx                     # Entry point React
│   │   └── styles.css                    # Estilos globais
│   └── index.html                        # HTML base

```

```
|   ├── package.json           # Dependências do frontend
|   ├── tsconfig.json         # Configuração TypeScript
|   └── vite.config.ts         # Configuração Vite
|
|── docker-compose.yml         # Configuração Docker
|── package.json               # Workspace root (monorepo)
|── .gitignore                 # Arquivos ignorados pelo Git
|── README.md                  # Documentação básica
```

## Modelo de Dados

O sistema utiliza **Prisma ORM** para gerenciar o modelo de dados. O schema principal está em [api/prisma/schema.prisma](#).

### Entidades Principais

#### 1. Employee (Funcionário)

Entidade central do sistema que representa um funcionário.

```
model Employee {
  id          Int      @id @default(autoincrement())
  name        String
  matricula   String   @unique
  costCenter  String
  branch      String
  admissionDate DateTime @db.Date
  terminationDate DateTime? @db.Date

  voucherMarketExcluded Boolean @default(false)
  voucherMealExcluded   Boolean @default(false)

  // Relacionamentos
  voucherMarketAllocations VoucherMarketAllocation[]
  voucherMealAllocations  VoucherMealAllocation[]
  unimedUsages             UnimedUsage[]
  medicalConvenioUsages    MedicalConvenioUsage[]

  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt

  @@index([branch])
  @@index([costCenter])
  @@index([terminationDate])
}
```

#### Campos:

- **id**: Identificador único (auto-incremento)

- **name**: Nome completo do funcionário
- **matricula**: Matrícula única (não pode ser alterada após criação)
- **costCenter**: Centro de custo
- **branch**: Filial (ex: "1", "2")
- **admissionDate**: Data de admissão
- **terminationDate**: Data de demissão (nullable para funcionários ativos)
- **voucherMarketExcluded**: Flag para excluir do Vale Mercado
- **voucherMealExcluded**: Flag para excluir do Vale Refeição

## 2. VoucherMarketInvoice (Fatura Vale Mercado)

Representa uma fatura mensal de Vale Mercado.

```
model VoucherMarketInvoice {
  id          Int          @id @default(autoincrement())
  competence  DateTime     @db.Date // 1º dia do mês
  invoiceNumber String
  invoiceValue Decimal     @db.Decimal(10, 2)
  status      VoucherInvoiceStatus @default(DRAFT)
  closedAt    DateTime?

  allocations  VoucherMarketAllocation[]

  @@unique([competence])
}
```

### Status:

- **DRAFT**: Rascunho (pode ser editado)
- **CLOSED**: Fechado (não pode mais ser editado)

## 3. VoucherMarketAllocation (Rateio Vale Mercado)

Rateio do Vale Mercado por funcionário.

```
model VoucherMarketAllocation {
  id          Int          @id @default(autoincrement())
  invoiceId   Int
  employeeId  Int
  amount      Decimal     @db.Decimal(10, 2)
  status      VoucherMarketAllocationStatus @default(DEFAULT)
  note        String?

  invoice     VoucherMarketInvoice @relation(...)
  employee    Employee              @relation(...)

  @@unique([invoiceId, employeeId])
}
```

**Status de Alocação:**

- **DEFAULT:** Valor padrão (R\$ 541,00)
- **FALTA:** Valor zero (falta no mês)
- **PROPORCIONAL:** Valor editável (proporcional)
- **EXCLUIDO:** Excluído do rateio (valor zero)

**Regra de Rateio:**

- **95%** empresa
- **5%** funcionários (rateado entre os participantes)

**4. VoucherMealInvoice (Fatura Vale Refeição)**

Representa uma fatura mensal de Vale Refeição. A empresa recebe **duas notas** por mês:

- **2ª quinzena** do mês (competência)
- **1ª quinzena** do mês seguinte

```
model VoucherMealInvoice {
  id                Int                @id @default(autoincrement())
  competence        DateTime           @db.Date // 1º dia do mês
  branch            String              @default("1")

  invoiceSecondHalfNumber String // Nº nota 2ª quinzena
  invoiceFirstHalfNextNumber String // Nº nota 1ª quinzena seguinte

  invoiceSecondHalf    Decimal @db.Decimal(12, 2) // Total 2ª quinzena
  invoiceFirstHalfNext Decimal @db.Decimal(12, 2) // Total 1ª quinzena seguinte

  status              VoucherInvoiceStatus @default(DRAFT)
  closedAt            DateTime?

  allocations          VoucherMealAllocation[]
  lines                VoucherMealInvoiceLine[]

  @@unique([competence, branch])
}
```

**5. VoucherMealInvoiceLine (Linhas da Fatura Vale Refeição)**

Itens detalhados da fatura (almoço, café, terceiros, etc.).

```
model VoucherMealInvoiceLine {
  id          Int                @id @default(autoincrement())
  invoiceId   Int
  part        VoucherMealInvoicePart // SECOND_HALF ou FIRST_HALF_NEXT
  kind        VoucherMealLineKind    // Tipo de item
}
```

```

    amount    Decimal    @db.Decimal(12, 2)

    @@unique([invoiceId, part, kind])
}

```

### Tipos de Itens (**VoucherMealLineKind**):

- **MEAL\_LUNCH**: Almoço (colaboradores)
- **MEAL\_LUNCH\_THIRD\_PARTY**: Almoço terceiros
- **MEAL\_LUNCH\_VISITORS**: Almoço visitantes
- **MEAL\_LUNCH\_DONATION**: Almoço doação
- **COFFEE\_SANDWICH**: Café/lanches (sanduíche)
- **COFFEE\_COFFEE\_LITER**: Café (litro)
- **COFFEE\_COFFEE\_MILK\_LITER**: Café com leite (litro)
- **COFFEE\_MILK\_LITER**: Leite (litro)
- **SPECIAL\_SERVICE**: Serviço especial
- **COFFEE\_GENERAL**: Itens gerais (Filial 02)
- **MISC\_SODA**: Refrigerante (Filial 02)
- **MISC\_MEAL\_EVENT**: Evento refeição (Filial 02)

## 6. VoucherMealAllocation (Rateio Vale Refeição)

Rateio do Vale Refeição por funcionário.

```

model VoucherMealAllocation {
    id          Int          @id @default(autoincrement())
    invoiceId   Int
    employeeId  Int
    employee20   Decimal    @db.Decimal(12, 2) // 20% funcionário
    company80   Decimal    @db.Decimal(12, 2) // 80% empresa
    total100    Decimal    @db.Decimal(12, 2) // 100% total

    @@unique([invoiceId, employeeId])
}

```

### Regra de Rateio:

- **80%** empresa
- **20%** funcionário

## 7. UnimedInvoice (Fatura Unimed)

Fatura mensal do plano de saúde Unimed.

```

model UnimedInvoice {
    id          Int          @id @default(autoincrement())
    competence  DateTime     @db.Date
}

```

```

    invoiceNumber String          @default("")
    invoiceValue  Decimal          @db.Decimal(12, 2)
    status        VoucherInvoiceStatus @default(DRAFT)
    closedAt      DateTime?

    usages        UnimedUsage[]

    @@unique([competence])
}

```

## 8. UnimedUsage (Uso Unimed)

Lançamento de uso do plano de saúde por funcionário.

```

model UnimedUsage {
    id            Int            @id @default(autoincrement())
    invoiceId     Int
    employeeId    Int
    kind          UnimedUsageKind // PERSONAL ou WORK_ACCIDENT
    amountTotal   Decimal        @db.Decimal(12, 2) // 100%
    amountEmployee Decimal        @db.Decimal(12, 2) // Parte funcionário
    amountCompany Decimal        @db.Decimal(12, 2) // Parte empresa
    note          String?

    @@index([invoiceId, employeeId])
}

```

### Tipos de Uso:

- **PERSONAL**: Uso pessoal (rateio padrão)
- **WORK\_ACCIDENT**: Acidente de trabalho (100% empresa)

## 9. MedicalConvenioInvoice (Fatura Convênio Médico)

Fatura mensal de convênios médicos (Laboratórios/Clinicas).

```

model MedicalConvenioInvoice {
    id            Int            @id @default(autoincrement())
    provider      MedicalConvenioProvider
    competence     DateTime      @db.Date
    invoiceNumber String          @default("")
    invoiceValue  Decimal        @db.Decimal(12, 2)
    status        VoucherInvoiceStatus @default(DRAFT)
    closedAt      DateTime?

    usages        MedicalConvenioUsage[]

    @@unique([provider, competence])
}

```



Provedores (MedicalConvenioProvider):

- LABORATORIO\_SANTA\_CRUZ
- CENTRO\_DIAGNOSTICO\_CAPAO\_RASO
- POLICLINICA\_CAPAO\_RASO
- POLICLINICA\_MANSUR

10. MedicalConvenioUsage (Uso Convênio Médico)

Lançamento de uso de convênio médico por funcionário (mesma estrutura do UnimedUsage).

🔑 API Backend

Configuração do Servidor

O servidor Express está configurado em `api/src/server.ts`:

- **Porta:** 3333 (configurável via `PORT` no `.env`)
- **CORS:** Configurável via `CORS_ORIGIN` (padrão: `http://localhost:5173`)
- **Body Parser:** JSON habilitado
- **Health Check:** `GET /health`

Estrutura de Rotas

|      |                    |                         |
|------|--------------------|-------------------------|
| /api |                    |                         |
| —    | /employees         | # Funcionários          |
| —    | /voucher-market    | # Vale Mercado          |
| —    | /voucher-meal      | # Vale Refeição         |
| —    | /unimed            | # Plano de Saúde Unimed |
| —    | /medical-convenios | # Convênios Médicos     |

Endpoints Detalhados

Funcionários (/employees)

| Método | Endpoint                  | Descrição                                  |
|--------|---------------------------|--------------------------------------------|
| GET    | /employees                | Lista funcionários com filtros e paginação |
| GET    | /employees/:id            | Busca funcionário por ID                   |
| POST   | /employees                | Cria novo funcionário                      |
| PUT    | /employees/:id            | Atualiza funcionário (exceto matrícula)    |
| PATCH  | /employees/:id/terminate  | Registra demissão                          |
| PATCH  | /employees/:id/reactivate | Reativa funcionário (remove demissão)      |

**Query Parameters (GET /employees):**

- **status**: `active` | `inactive` | `all` (padrão: `active`)
- **search**: Busca por nome ou matrícula
- **branch**: Filtra por filial
- **costCenter**: Filtra por centro de custo
- **page**: Número da página (padrão: 1)
- **pageSize**: Itens por página (padrão: 20, máx: 200)

**Exemplo de Resposta:**

```
{
  "items": [
    {
      "id": 1,
      "name": "João Silva",
      "matricula": "001",
      "costCenter": "CC001",
      "branch": "1",
      "admissionDate": "2020-01-15T00:00:00.000Z",
      "terminationDate": null,
      "voucherMarketExcluded": false,
      "voucherMealExcluded": false
    }
  ],
  "total": 100,
  "page": 1,
  "pageSize": 20
}
```

**Validação de Dados:**

- Utiliza **Zod** para validação de schemas
- **admissionDate** e **terminationDate** aceitam formatos:
  - `YYYY-MM-DD` (ISO)
  - `YYYYMMDD` (compacto)
- **matricula** é única e não pode ser alterada após criação
- **terminationDate** não pode ser anterior a **admissionDate**

**Vale Mercado (/voucher-market)**

Endpoints para gestão de faturas e rateio do Vale Mercado.

**Principais Funcionalidades:**

- Criação de faturas por competência
- Rateio automático entre funcionários
- Controle de status (DRAFT/CLOSED)
- Exclusão de funcionários do rateio

## Vale Refeição (/voucher-meal)

Endpoints para gestão de faturas e rateio do Vale Refeição.

### Principais Funcionalidades:

- Suporte a duas filiais
- Gestão de duas notas por mês (2ª quinzena + 1ª quinzena seguinte)
- Rateio de diferentes tipos de itens (almoço, café, terceiros, etc.)
- Cálculo automático de rateio (80% empresa / 20% funcionário)

## Unimed (/unimed)

Endpoints para gestão de faturas e uso do plano de saúde Unimed.

### Principais Funcionalidades:

- Criação de faturas por competência
- Lançamento de usos por funcionário
- Diferenciação entre uso pessoal e acidente de trabalho
- Rateio automático (exceto acidente de trabalho = 100% empresa)

## Convênios Médicos (/medical-convenios)

Endpoints para gestão de múltiplos prestadores de serviços médicos.

### Provedores Suportados:

- Laboratório Santa Cruz
- Centro de Diagnóstico Capão Raso
- Policlínica Capão Raso
- Policlínica Mansur

### Funcionalidades:

- Gestão independente por provedor
- Mesma estrutura de rateio do Unimed
- Controle de faturas por competência

## Validação e Formatação

O sistema utiliza funções utilitárias em `api/src/lib/format.ts`:

- `parseDateFlexible()`: Converte strings de data para objetos Date
  - Aceita `YYYY-MM-DD` ou `YYYYMMDD`
- `normalizeName()`: Normaliza nomes (remove espaços duplicados)
- `normalizeSimple()`: Remove espaços em branco de strings simples

## Tratamento de Erros

- **400 Bad Request**: Dados inválidos (validação Zod)

- **404 Not Found:** Recurso não encontrado
  - **409 Conflict:** Conflito (ex: matrícula duplicada)
  - **500 Internal Server Error:** Erro interno do servidor
- 

## Frontend

### Arquitetura Frontend

O frontend é uma **SPA (Single Page Application)** construída com React e TypeScript.

### Roteamento

O sistema utiliza **hash-based routing** (sem biblioteca externa):

- `#/employees` → Página de Funcionários
- `#/vale-mercado` → Vale Mercado
- `#/vale-refeicao` → Vale Refeição Filial 01
- `#/vale-refeicao-filial-02` → Vale Refeição Filial 02
- `#/unimed` → Unimed
- `#/convenio-laboratorio-santa-cruz` → Laboratório Santa Cruz
- `#/convenio-centro-diagnostico-capao-raso` → Centro de Diagnóstico
- `#/convenio-policlinica-capao-raso` → Policlínica Capão Raso
- `#/convenio-policlinica-mansur` → Policlínica Mansur

### Componentes Principais

#### **App.tsx**

Componente raiz que gerencia:

- Estado da rota atual
- Navegação entre páginas
- Sidebar com menu de navegação
- Renderização condicional de páginas

#### **Sidebar**

Menu lateral com:

- Navegação principal
- Grupo expansível para "Convênios médicos"
- Indicador da página atual
- Título dinâmico baseado na rota

#### **Páginas**

Cada módulo possui sua própria página:

- `EmployeesPage`: CRUD de funcionários

- **ValeMercadoPage**: Gestão Vale Mercado
- **ValeRefeicaoPage**: Gestão Vale Refeição Filial 01
- **ValeRefeicaoFilial02Page**: Gestão Vale Refeição Filial 02
- **UnimedPage**: Gestão Unimed
- Páginas de convênios médicos (4 páginas)

## Cliente API

O frontend utiliza **Axios** para comunicação com a API:

```
// web/src/api.ts
export const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3333",
});
```

### Variáveis de Ambiente:

- **VITE\_API\_URL**: URL base da API (padrão: `http://localhost:3333`)

## Tipos TypeScript

Todos os tipos da API estão definidos em `web/src/api.ts`:

- **Employee**
- **VoucherMarketInvoice**, **VoucherMarketAllocation**
- **VoucherMealInvoice**, **VoucherMealAllocation**
- **UnimedInvoice**, **UnimedUsage**
- **MedicalConvenioInvoice**, **MedicalConvenioUsage**

## Estilos

O sistema utiliza **CSS puro** (sem frameworks CSS):

- `web/src/styles.css`: Estilos globais
- `web/src/styles/app.css`: Estilos da aplicação
- Classes utilitárias para layout e componentes

---

## Configuração e Instalação

### Pré-requisitos

- **Node.js** 18 ou superior
- **Docker** e **Docker Compose** (recomendado para MySQL)
- **npm** ou **yarn**

### Passo a Passo

#### 1. Clonar o Repositório

```
git clone <url-do-repositorio>
cd rh-gestao-funcionarios
```

## 2. Subir o Banco de Dados (MySQL + Adminer)

Na raiz do projeto:

```
docker compose up -d
```

Isso inicia:

- **MySQL** na porta **3308** (container: **rh\_mysql**)
- **Adminer** na porta **8080** (container: **rh\_adminer**)

### Acesso ao Adminer:

- URL: `http://localhost:8080`
- Sistema: MySQL
- Servidor: **mysql**
- Usuário: **root**
- Senha: **root**
- Base de dados: **rh**

## 3. Configurar a API

```
cd api
cp .env.example .env
```

Edite o arquivo **.env** se necessário:

```
DATABASE_URL="mysql://root:root@localhost:3308/rh"
PORT=3333
CORS_ORIGIN="http://localhost:5173"
SEED_JSON_PATH="./data/funcionarios.json"
```

## 4. Instalar Dependências e Configurar Banco

```
npm install
npx prisma generate
npx prisma migrate dev --name init
npm run seed
```

**Comandos explicados:**

- `npm install`: Instala dependências
- `npx prisma generate`: Gera o cliente Prisma
- `npx prisma migrate dev`: Aplica migrações e cria o banco
- `npm run seed`: Popula o banco com dados iniciais

**5. Iniciar a API**

```
npm run dev
```

A API estará disponível em: <http://localhost:3333>

**Health Check:** <http://localhost:3333/health>

**6. Configurar e Iniciar o Frontend**

Em outro terminal:

```
cd web  
npm install  
npm run dev
```

O frontend estará disponível em: <http://localhost:5173>

**Scripts Disponíveis****Backend (`api/package.json`)**

| Script                               | Descrição                                            |
|--------------------------------------|------------------------------------------------------|
| <code>npm run dev</code>             | Inicia servidor em modo desenvolvimento (hot reload) |
| <code>npm run build</code>           | Compila TypeScript para JavaScript                   |
| <code>npm run start</code>           | Inicia servidor em produção                          |
| <code>npm run seed</code>            | Executa seed do banco de dados                       |
| <code>npm run prisma:generate</code> | Gera cliente Prisma                                  |
| <code>npm run prisma:migrate</code>  | Executa migrações                                    |

**Frontend (`web/package.json`)**

| Script                   | Descrição                               |
|--------------------------|-----------------------------------------|
| <code>npm run dev</code> | Inicia servidor de desenvolvimento Vite |

| Script                       | Descrição                    |
|------------------------------|------------------------------|
| <code>npm run build</code>   | Gera build de produção       |
| <code>npm run preview</code> | Preview do build de produção |

## Detalhes Técnicos

### Banco de Dados

#### Configuração MySQL

- **Porta:** 3308 (mapeada do container)
- **Usuário:** root
- **Senha:** root
- **Database:** rh
- **Charset:** utf8mb4 (padrão MySQL 8.0)

#### Migrações Prisma

As migrações estão em `api/prisma/migrations/` e incluem:

1. `20260108173749_init`: Migração inicial
2. `20260109114320_add_vale_mercado`: Adição de Vale Mercado
3. `20260116123546_add_vale_refeicao`: Adição de Vale Refeição
4. `20260129180000_add_unimed`: Adição de Unimed
5. `20260202193000_add_medical_convenios`: Adição de Convênios Médicos
6. E outras migrações de ajustes

#### Seed do Banco

O seed (`api/prisma/seed.ts`) lê o arquivo `api/data/funcionarios.json` e:

- Cria funcionários que não existem
- Atualiza funcionários existentes (baseado na matrícula)
- Mantém a integridade dos dados

#### Formato do JSON de seed:

```
[
  {
    "Nome": "João Silva",
    "Matricula": "001",
    "Centro Custo": "CC001",
    "Admissao": "20200115",
    "Demissao": "",
    "Filial": "1"
  }
]
```



## Segurança

### CORS

O CORS está configurado para permitir apenas origens específicas:

```
app.use(cors({
  origin: process.env.CORS_ORIGIN?.split(",").map(s => s.trim()) ?? "*"
}));
```

**Recomendação:** Em produção, configure `CORS_ORIGIN` com as URLs permitidas.

### Validação de Dados

- **Backend:** Zod para validação de schemas
- **Frontend:** Validação básica antes de enviar requisições
- **Banco:** Constraints do Prisma (unique, foreign keys)

## Performance

### Índices do Banco

O schema Prisma define índices estratégicos:

- **Employee:** `branch`, `costCenter`, `terminationDate`, `voucherMarketExcluded`, `voucherMealExcluded`
- **VoucherMarketInvoice:** `competence` (unique), `invoiceNumber`
- **VoucherMealInvoice:** `competence` + `branch` (unique), `branch`
- Relacionamentos: índices em foreign keys

### Paginação

A API de funcionários implementa paginação:

- Padrão: 20 itens por página
- Máximo: 200 itens por página
- Query parameters: `page` e `pageSize`

## Tratamento de Erros

### Backend

- Validação com Zod retorna erros estruturados
- Erros do Prisma são tratados (ex: P2002 para duplicatas)
- Mensagens de erro em português

### Frontend

- Tratamento de erros HTTP com Axios
- Feedback visual para o usuário
- Validação de formulários

## Desenvolvimento

### Hot Reload

- **Backend:** `ts-node-dev` com `--respawn` e `--transpile-only`
- **Frontend:** Vite HMR (Hot Module Replacement)

### TypeScript

- **Strict mode:** Habilitado
- **Target:** ES2020+
- **Module:** ESM (frontend) / CommonJS (backend)

## Monorepo

O projeto utiliza **npm workspaces**:

```
{  
  "workspaces": [ "api", "web" ]  
}
```

Isso permite:

- Gerenciamento centralizado de dependências
- Compartilhamento de código entre projetos
- Scripts unificados



## Fluxos de Trabalho

### Fluxo de Cadastro de Funcionário

1. Usuário acessa `/employees`
2. Clica em "Novo Funcionário"
3. Preenche formulário (nome, matrícula, centro de custo, filial, data admissão)
4. Frontend valida dados
5. Envia `POST /employees` para API
6. API valida com Zod
7. Prisma cria registro no banco
8. Retorna funcionário criado
9. Frontend atualiza lista

### Fluxo de Demissão

1. Usuário seleciona funcionário
2. Clica em "Demitir"
3. Informa data de demissão
4. Frontend envia `PATCH /employees/:id/terminate`
5. API valida data (não pode ser anterior à admissão)
6. Prisma atualiza `terminationDate`
7. Funcionário passa a aparecer como "inativo"

## Fluxo de Rateio Vale Mercado

1. Usuário acessa `/vale-mercado`
2. Seleciona competência (mês)
3. Cria nova fatura ou edita existente
4. Informa número e valor da nota
5. Sistema calcula rateio automático:
  - Lista funcionários ativos não excluídos
  - Distribui valor padrão (R\$ 541,00) ou proporcional
6. Usuário pode ajustar valores individuais
7. Sistema calcula totais:
  - Soma dos rateios
  - 95% empresa
  - 5% funcionários
8. Usuário fecha fatura (status CLOSED)

## Fluxo de Rateio Vale Refeição

1. Usuário acessa `/vale-refeicao` (Filial 01 ou 02)
2. Seleciona competência
3. Cria/edita fatura
4. Informa valores das duas notas:
  - 2ª quinzena do mês
  - 1ª quinzena do mês seguinte
5. Lança itens detalhados (almoço, café, terceiros, etc.)
6. Sistema calcula rateio:
  - Almoço: rateado entre funcionários (80% empresa / 20% funcionário)
  - Café: rateado igualmente entre funcionários
  - Terceiros: apenas relatório (não rateado)
7. Usuário fecha fatura

## Fluxo de Gestão Unimed

1. Usuário acessa `/unimed`
2. Seleciona competência
3. Cria/edita fatura
4. Informa número e valor da nota
5. Lança usos por funcionário:
  - Tipo: pessoal ou acidente de trabalho
  - Valor total do procedimento

6. Sistema calcula rateio:
    - Pessoal: rateio padrão (funcionário/empresa)
    - Acidente: 100% empresa
  7. Usuário fecha fatura
- 

## Notas Adicionais

### Convenções de Código

- **Backend:** TypeScript com tipos explícitos
- **Frontend:** TypeScript com componentes funcionais React
- **Nomenclatura:**
  - Variáveis: camelCase
  - Componentes: PascalCase
  - Arquivos: PascalCase (componentes), camelCase (utilitários)

### Próximos Passos Sugeridos

- ☐ Autenticação e autorização
- ☐ Logs estruturados
- ☐ Testes unitários e de integração
- ☐ Documentação da API (Swagger/OpenAPI)
- ☐ Deploy automatizado (CI/CD)
- ☐ Backup automático do banco
- ☐ Dashboard com métricas e gráficos

### Limitações Conhecidas

- Sem autenticação/autorização
  - Sem validação de permissões
  - Sem sistema de auditoria/logs
  - Sem cache de consultas
  - Sem rate limiting na API
- 

## Suporte

Para dúvidas ou problemas:

1. Consulte esta documentação
  2. Verifique os logs do servidor
  3. Consulte o código-fonte
  4. Entre em contato com a equipe de desenvolvimento
- 

**Última atualização:** Fevereiro 2026